

État de l'art

PER2025-057 - Développement

Systèmes collaboratifs pour le contrôle d'atterrissage d'un nano-drone sur plateforme mobile

Étudiants: Komi Jean Paul ASSIMPAH (SI5 IoT-CPS),
Alban FALCOZ (SI5 IA-ID),
Evan GALLI (SI5 IoT-CPS),
Alexandre GRIPARI (SI5 IA-ID)

Encadrant: Gérald ROCHER

I. Introduction à la problématique

L'utilisation de véhicules aériens sans pilote (UAV) connaît une expansion rapide dans des secteurs variés tels que l'inspection industrielle, la recherche et le sauvetage, ou la livraison logistique. Si le pilotage manuel ou assisté par GPS est maîtrisé, l'autonomie complète dans des environnements complexes reste un défi ouvert. Une tâche particulièrement critique est l'atterrissage autonome sur des plateformes mobiles (par exemple, un véhicule terrestre ou un navire), qui exige une précision de contrôle et une robustesse aux perturbations que les méthodes classiques peinent à offrir simultanément.

Les approches de contrôle traditionnelles, telles que le PID (Proportionnel-Intégral-Dérivé) ou le MPC (Model Predictive Control), dépendent fortement de la précision du modèle dynamique du robot et de l'estimation d'état. Elles manquent souvent de flexibilité face à des situations imprévues, comme des pannes de capteurs ou des turbulences aérodynamiques non modélisées. C'est dans ce contexte que l'apprentissage par renforcement profond (Deep Reinforcement Learning - DeepRL) émerge comme une alternative disruptive. Le DeepRL permet à un agent d'apprendre une politique de contrôle « de bout en bout » (end-to-end), mappant directement les observations brutes (images, données inertielles) aux commandes moteurs, sans nécessiter d'ingénierie manuelle complexe des caractéristiques.

Cependant, l'application du DeepRL à la robotique physique se heurte à une barrière majeure : le coût et le danger de l'entraînement en conditions réelles. Les algorithmes de DeepRL nécessitent des millions d'interactions pour converger, ce qui est impraticable sur un drone réel en raison de la fragilité du matériel et de la durée limitée des batteries. L'entraînement se fait donc quasi exclusivement en simulation. Cette stratégie introduit le

problème central de cette étude : le « Reality Gap » (fossé de réalité). Ce fossé désigne la divergence entre les distributions de données générées par le simulateur et celles rencontrées dans le monde réel physique. Si ce fossé n'est pas comblé, une politique performante en simulation peut échouer catastrophiquement dans la réalité.

Cet état de l'art analyse les avancées récentes dans l'application du DeepRL au contrôle de drones, en se concentrant spécifiquement sur les architectures algorithmiques, les environnements de simulation nouvelle génération, et les stratégies de transfert Sim-to-Real (notamment la Domain Randomization) pour surmonter le Reality Gap.

II. Approches de DeepRL pour le contrôle de drones

Architectures Algorithmiques pour le Contrôle Robotique (voir quel titre on garde)

L'application de l'apprentissage par renforcement au contrôle de drones a suivi une évolution marquée, passant d'approches discrètes à des méthodes continues plus sophistiquées, pour converger vers des standards actuels privilégiant la robustesse.

II.1. Les premières approches : L'ère du Discret (DQN)

Les premiers travaux significatifs dans le domaine, notamment ceux de Polvara et al., se sont appuyés sur le Deep Q-Network (DQN) [1]. Dans leurs recherches sur l'atterrissage autonome, ils ont utilisé une hiérarchie de réseaux DQN pour gérer des sous-tâches distinctes comme la détection de marqueurs au sol et la manœuvre de descente verticale [1]. Cependant, le DQN est conçu pour des espaces d'actions discrets, ce qui a obligé les auteurs à discrétiser les commandes du drone (ex: avancer, reculer, descendre, stop) [1]. Bien que cette méthode ait permis des avancées pionnières en apprenant des politiques de haut niveau sans supervision humaine, elle impose des limitations intrinsèques pour le pilotage fin nécessaire en robotique, induisant des mouvements saccadés moins adaptés aux dynamiques complexes [1].

II.2. Transition vers le Contrôle Continu (DDPG et TD3)

Pour surmonter les limites des actions discrètes, la recherche s'est orientée vers des algorithmes capables de gérer des espaces d'actions continus, essentiels pour un contrôle fluide. Rodriguez-Ramos et al. ont démontré l'efficacité du Deep Deterministic Policy Gradient (DDPG) [2]. Basé sur l'architecture Actor-Critic, DDPG permet de mapper directement des états continus (ou des images brutes) vers des commandes moteurs continues [2].

Néanmoins, DDPG souffre de biais d'estimation, notamment la surestimation des valeurs Q. Pour y remédier, des algorithmes plus récents comme TD3 (Twin Delayed DDPG) ont été introduits. Wang et al. ont comparé DDPG, TD3 et SAC (Soft Actor-Critic) pour des tâches d'atterrissage collaboratif [3]. Leurs résultats ont montré que TD3, grâce à l'utilisation de deux réseaux critiques (« twin critics ») pour réduire la surestimation et au retardement de la mise à jour de la politique, offrait une meilleure stabilité et précision que SAC dans leur scénario spécifique [3]. Dans leurs expériences, SAC a montré de moins bonnes performances initiales, se concentrant excessivement sur l'exploration au détriment de l'accumulation de récompenses [3].

II.3. L'État de l'Art actuel : Proximal Policy Optimization (PPO)

Aujourd'hui, l'algorithme PPO (Proximal Policy Optimization) s'est imposé comme le standard de facto pour la robotique mobile. Schwarke et al. (auteurs de la librairie RSL-RL)

soulignent que PPO est devenu l'algorithme par défaut en apprentissage robotique en raison de sa simplicité d'implémentation et de sa robustesse face aux hyperparamètres [4]. Contrairement à DDPG ou TD3 qui sont « off-policy », PPO est une méthode « on-policy » qui apprend directement de la politique courante, ce qui simplifie la gestion des données et le rend particulièrement robuste pour les tâches de locomotion et de navigation [4].

Des travaux récents, comme ceux de Aikins et al., utilisent PPO comme baseline de performance, le qualifiant d'algorithme « state-of-the-art » pour sa stabilité, sa fiabilité et son efficacité d'échantillonnage [5]. Bien que PPO reste le socle algorithmique privilégié pour l'apprentissage de la locomotion et de la navigation de drones en environnements complexes, certaines limites apparaissent dans des conditions d'observabilité partielle sévère. Par exemple, Aikins et al. notent que si PPO performe bien avec un bruit modéré, ses performances chutent drastiquement (de 82% à 19% de succès) lorsque les capteurs subissent des pertes de signal intermittentes (« flicker »), là où des variantes intégrant de la mémoire (comme les réseaux LSTM) maintiennent une robustesse supérieure [5].

III. Stratégies pour surmonter le « Reality Gap »

Transfert Sim-to-Real et Comblement du Reality Gap (voir quel titre on garde)

Le transfert d'une politique apprise en simulation vers un robot physique est l'un des défis les plus critiques du Deep Reinforcement Learning (DeepRL). Ce défi, connu sous le nom de « Reality Gap », provient des divergences inévitables entre le modèle simulé et la réalité physique, tant au niveau de la dynamique du vol que de la perception sensorielle. L'analyse de la littérature récente permet d'identifier trois stratégies majeures pour surmonter cet obstacle : la « Domain Randomization », la gestion des latences via l'architecture du réseau, et l'apprentissage par curriculum.

III.1. Domain Randomization (Aléatorisation du Domaine)

La méthode la plus répandue pour rendre l'agent plus robuste face aux incertitudes du monde réel est la Domain Randomization. L'idée centrale est d'entraîner l'agent dans une variété d'environnements simulés aux propriétés physiques légèrement différentes, de sorte que le monde réel ne soit perçu que comme une variation supplémentaire de l'entraînement.

Aikins et al. ont démontré l'efficacité de l'ajout de bruit gaussien et de « flickering » (perte intermittente de signal) sur les observations simulées. Leur approche, testée sur un simulateur haute fidélité (Isaac Gym, ancêtre d'Isaac Lab), a permis d'obtenir des taux d'atterrissage réussis supérieurs à ceux des méthodes classiques (Lee-EKF) et de l'algorithme PPO standard en présence de bruit [5]. De même, Do et al. injectent du bruit dans les observations de position, d'orientation et de vitesse lors de l'entraînement pour imiter les imperfections des capteurs réels comme le système VICON [6].

Pour garantir la robustesse face au vent ou aux effets de sol, Wang et al. intègrent des perturbations de vent sous forme de bruit gaussien continu directement dans les commandes de manœuvre du drone durant l'entraînement [3].

III.2. Gestion des Latences et Architecture Réseau

Une cause majeure d'échec du transfert Sim2Real sur des nano-drones comme le Crazyflie est la latence inhérente aux actionneurs et à la dynamique des moteurs, souvent simplifiée en simulation.

Do et al. soulignent que les moteurs du Crazyflie présentent un comportement passe-bas significatif, créant un délai entre la commande et la réponse physique [6]. Pour combler ce fossé, ils proposent d'inclure l'historique des actions passées (les commandes moteurs précédentes) directement dans le vecteur d'observation fourni au réseau neuronal. Cette technique permet au réseau d'inférer implicitement les délais du système.

Pour pallier l'observabilité partielle (due par exemple à des pannes de capteurs ou des occlusions visuelles), l'intégration de cellules de mémoire de type LSTM (Long Short-Term Memory) dans l'architecture de l'agent s'avère efficace. L'architecture RPO-LSTM proposée par Aikins et al. permet de capturer les dépendances temporelles à long terme, rendant le contrôle robuste même lorsque les données des capteurs sont bruitées ou manquantes [5].

III.3. Fidélité Visuelle et Physique de la Simulation

La qualité du simulateur joue un rôle prépondérant pour réduire le fossé de réalité a priori.

Le framework Pegasus Simulator, construit sur Isaac Sim, démontre l'importance d'un environnement photo-réaliste pour valider des algorithmes de contrôle et de perception complexes avant le déploiement réel [7]. Contrairement aux simulateurs classiques comme Gazebo, ces outils modernes permettent de simuler des capteurs visuels avec une fidélité proche du réel, facilitant le transfert des politiques basées sur la vision.

Bien que la randomisation soit puissante, Kooi et Babuška notent qu'une identification précise des paramètres du système (comme les courbes de poussée des moteurs) reste cruciale [8]. Dans leurs travaux sur l'atterrissage incliné avec un Crazyflie 2.1, ils ont réussi le transfert Sim2Real sans randomisation excessive, simplement en ajustant finement le modèle de poussée statique (PWM to Thrust) pour correspondre au matériel réel.

III.4. Apprentissage par Curriculum (ACL)

Enfin, pour éviter que l'agent n'apprenne des comportements aberrants en début d'entraînement (ce qui creuse le fossé avec la réalité), Wang et al. proposent une méthode d'Automatic Curriculum Learning [3]. L'entraînement commence par des tâches simples (atterrissage stationnaire) et augmente progressivement la difficulté (cible mobile, vent, accélérations brusques). Cette approche structure l'apprentissage et produit des politiques plus stables et transférables que l'entraînement direct sur des scénarios complexes.

IV. Discussion et Synthèse Critique

L'analyse croisée révèle deux tendances majeures :

1. **L'avènement des simulateurs GPU** : L'adoption d'outils comme **NVIDIA Isaac Sim** permet une simulation physique accélérée et photoréaliste, réduisant drastiquement les temps d'entraînement.
2. **La mémoire comme outil d'adaptation** : Au-delà de la gestion du bruit, les architectures récurrentes (LSTM) agissent comme un méta-apprentissage implicite, adaptant la politique en temps réel aux dynamiques physiques.

Cependant, des défis persistent :

- **Corrélations fallacieuses** : Les agents peuvent apprendre des comportements « superstitieux » (lier une erreur de tangage à une commande de roulis sans raison physique).
- **Le dilemme Nominal vs Panne** : Un agent entraîné pour gérer des pannes moteurs devient souvent moins performant en vol nominal.

- **Interactions physiques complexes** : L'effet de sol et les contacts restent difficiles à modéliser parfaitement, justifiant l'approche hybride (**System ID + Domain Randomization**).

V. Bibliographie

- [1] R. Polvara *et al.*, « Autonomous Quadrotor Landing using Deep Reinforcement Learning ». [En ligne]. Disponible sur: <https://arxiv.org/abs/1709.03339>
- [2] A. Rodriguez-Ramos, C. Sampedro, H. Bavle, P. de la Puente, et P. Campoy, « A Deep Reinforcement Learning Strategy for UAV Autonomous Landing on a Moving Platform », *Journal of Intelligent & Robotic Systems*, vol. 93, n° 1, p. 351- 366, févr. 2019, doi: 10.1007/s10846-018-0891-8.
- [3] C. Wang, J. Wang, C. Wei, Y. Zhu, D. Yin, et J. Li, « Vision-Based Deep Reinforcement Learning of UAV-UGV Collaborative Landing Policy Using Automatic Curriculum », *Drones*, vol. 7, n° 11, 2023, doi: 10.3390/drones7110676.
- [4] C. Schwarke, M. Mittal, N. Rudin, D. Hoeller, et M. Hutter, « RSL-RL: A Learning Library for Robotics Research ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2509.10771>
- [5] G. Aikins, S. Jagtap, et K.-D. Nguyen, « A Robust Strategy for UAV Autonomous Landing on a Moving Platform under Partial Observability », *Drones*, vol. 8, n° 6, 2024, doi: 10.3390/drones8060232.
- [6] T.-D. Do, N. X. Mung, et S. K. Hong, « Deep Reinforcement Learning-based Quadcopter Controller: A Practical Approach and Experiments ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2406.08815>
- [7] M. Jacinto *et al.*, « Pegasus Simulator: An Isaac Sim Framework for Multiple Aerial Vehicles Simulation », in *2024 International Conference on Unmanned Aircraft Systems (ICUAS)*, IEEE, juin 2024, p. 917- 922. doi: 10.1109/icuas60882.2024.10556959.
- [8] J. E. Kooi et R. Babuška, « Inclined Quadrotor Landing using Deep Reinforcement Learning », in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, sept. 2021, p. 2361- 2368. doi: 10.1109/IROS51168.2021.9636096.
- [9] D. Falanga, A. Zanchettin, A. Simovic, J. Delmerico, et D. Scaramuzza, « Vision-based autonomous quadrotor landing on a moving platform », in *2017 IEEE International Symposium on Safety, Security and Rescue Robotics (SSRR)*, oct. 2017, p. 200- 207. doi: 10.1109/SSRR.2017.8088164.
- [10] J. Huber, F. Hélénon, H. Watrelot, F. B. Amar, et S. Doncieux, « Domain Randomization for Sim2real Transfer of Automatically Generated Grasping Datasets ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2310.04517>
- [11] L. Weng, « Domain Randomization for Sim2Real Transfer », *lilianweng.github.io*, 2019, [En ligne]. Disponible sur: <https://lilianweng.github.io/posts/2019-05-05-domain-randomization/>
- [12] X. Chen, J. Hu, C. Jin, L. Li, et L. Wang, « Understanding Domain Randomization for Sim-to-real Transfer ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2110.03239>
- [13] M. Kulkarni, T. J. L. Forgaard, et K. Alexis, « Aerial Gym – Isaac Gym Simulator for Aerial Robots ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2305.16510>

- [14] A. Gautam, M. Singh, P. B. Sujit, et S. Saripalli, « Autonomous Quadcopter Landing on a Moving Target », *Sensors*, vol. 22, n° 3, 2022, doi: 10.3390/s22031116.
- [15] K. Guo, P. Tang, H. Wang, D. Lin, et X. Cui, « Autonomous Landing of a Quadrotor on a Moving Platform via Model Predictive Control », *Aerospace*, vol. 9, n° 1, 2022, doi: 10.3390/aerospace9010034.
- [16] P. Ladosz, M. Mammadov, H. Shin, W. Shin, et H. Oh, « Autonomous Landing on a Moving Platform Using Vision-Based Deep Reinforcement Learning », *IEEE Robotics and Automation Letters*, vol. 9, n° 5, p. 4575- 4582, mai 2024, doi: 10.1109/LRA.2024.3379837.
- [17] N. Bernini *et al.*, « Reinforcement learning with formal performance metrics for quadcopter attitude control under non-nominal contexts », *Engineering Applications of Artificial Intelligence*, vol. 127, oct. 2023, doi: 10.1016/j.engappai.2023.107090.

VI. Annexe : Plan d'Avancement du Projet

A voir si on en fait un à nouveau